

6CM504 Concurrency and communication

Modelling and verification

Prof. Alistair A. McEwan

School of Computing
University of Derby

Table of Contents

- 1 Introduction
- 2 Algebra, laws, and semantics
- 3 But what about verification?
- 4 The semantics of CSP
- 5 Algebraic laws
- 6 Summary

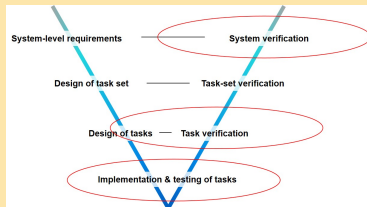
This week

So far in this module, we have seen the basic operators of CSP, and some examples constructed using them. We were able to witness properties of our models by inspection—but this is not going to be possible as the size and complexities of our models grow. In this lecture, we will look at one of the fundamentally important aspects of CSP—the algebra.

Table of Contents

- 1 Introduction
- 2 Algebra, laws, and semantics**
- 3 But what about verification?
- 4 The semantics of CSP
- 5 Algebraic laws
- 6 Summary

Why are we interested?



- 1 Understanding models requires having a well-defined understanding of what they mean
- 2 Verification typically happens at higher levels of the V model
- 3 We shall now look at algebra and semantics
- 4 This will allow us to define clear meanings of the relationships between different processes

Testing versus verification

- ① It is commonly accepted that testing does not give full assurance about the behaviours (and safety) of a system
 - You cannot give assurances about non-deterministic systems
- ② Even simple protocols can have serious (hidden) flaws
 - Security, communications, resource sharing
 - Modelling can enable deeper understanding
- ③ Formal models of systems allow rigorous investigations
 - Exhaustive state-space searches, theorem proving
- ④ Help with development, as well as prototyping
- ⑤ Becoming accepted as development and verification technologies
 - DO178C

Table of Contents

- 1 Introduction
- 2 Algebra, laws, and semantics
- 3 But what about verification?**
- 4 The semantics of CSP
- 5 Algebraic laws
- 6 Summary

(Really!) Understanding CSP

- ❶ So far we have built several models of processes
 - But we are not able to say much about them
 - Or make promises about their behaviour
- ❷ In today's lecture we will study how to extract meaning from our models
 - Different techniques for extracting meaning from processes
 - Ways of relating models to each other
 - How to use these relationships to make promises

What does it mean to prove one process is equal to another?

Table of Contents

- 1 Introduction
- 2 Algebra, laws, and semantics
- 3 But what about verification?
- 4 The semantics of CSP**
- 5 Algebraic laws
- 6 Summary

Remember this C code?

```
❶ int a;  
❷ void Task1() {  
❸     int x;  
❹     x= a;  
❺     movingAverage= MovingAverage(x);  
❻     a= movingAverage;  
❼ }
```

Three approaches to CSP meaning

1 Algebraic

- Sets of laws that processes must satisfy
- This is where the term “process algebra” comes from

2 Behavioural

- Process observations as traces
- These form the basis of refinement relations

3 Operational

- Describes how the execution of a process may proceed using transition laws

4 Each of these are different rigorous mathematical approaches to giving meaning to the language of CSP

- In this module we will study behavioural models and some algebraic laws

Table of Contents

- 1 Introduction
- 2 Algebra, laws, and semantics
- 3 But what about verification?
- 4 The semantics of CSP
- 5 Algebraic laws**
- 6 Summary

What does algebraic mean?

- ① An *algebraic law* is a statement that two expressions (involving some operators and identifiers representing arbitrary processes) are equal
- ② By equal, we mean that the two sides are the same
- ③ For CSP this means that their communicating behaviours are indistinguishable from each other by the environment

What does algebraic mean?

1 Example algebraic laws that you will be familiar with

$x + y = y + x$	a commutative law from arithmetic
$x \cup y = y \cup x$	a commutative law from set theory
$(x + y) + z = x + (y + z)$	an associative law from arithmetic
$0 + x = x$	a unit law from arithmetic
$x.(y + z) = x.y + x.z$	a distributive law from boolean logic
$\emptyset \cup x = \emptyset$	a zero law from set theory
$x \cup x = x$	an idempotence law from set theory

Some algebraic laws in CSP

- 1 Idempotence: it doesn't matter how many times we place a process in choice with itself, it remains unchanged

$$P \quad [] \quad P = P \qquad ([] - idem)$$

$$P \quad | \sim | \quad P = P \qquad (| \sim | - idem)$$

- 2 Note: we will give our laws names, so we can refer to them later

Some algebraic laws in CSP

- 1 Symmetry: we can swap the sides of the operator that a process appears on—the operator is symmetric

$$P \parallel Q = Q \parallel P \quad (\parallel - \text{sym})$$

$$P \mid\sim\mid Q = Q \mid\sim\mid P \quad (\mid\sim\mid - \text{sym})$$

Some algebraic laws in CSP

- 1 Associativity: processes can be bracketed together, and the brackets can be moved

$$(P \parallel Q) \parallel R = P \parallel (Q \parallel R) \quad (\parallel - \text{assoc})$$

$$(P \mid \sim \mid Q) \mid \sim \mid R = P \mid \sim \mid (Q \mid \sim \mid R) \quad (\mid \sim \mid - \text{assoc})$$

Example: equivalent processes

① Using these laws, if I assume I have defined 3 processes P, Q, and R, I can make some valuable statements

② For instance assume the process A is defined as

$$A = (P \mid \sim \mid Q) \parallel R$$

③ And the process B is defined as

$$B = R \parallel (Q \mid \sim \mid P) \parallel R$$

④ Then I can assert that $A=B$

- That is, they are actually the same process
- If a customer asks me to give them A, and I gave them B, they would not be able to tell the difference

⑤ Question: what laws have I used to show that $A=B$?

Some distributive laws

- 1 These laws talk about how some operators distribute through other operators

- You will have seen comparable examples in boolean logic

$$P \text{ [] } (Q \mid \sim \mid R) = (P \text{ [] } Q) \mid \sim \mid (P \text{ [] } R) \quad ([\text{]} - \text{dist})$$

$$a \rightarrow (P \mid \sim \mid Q) = (a \rightarrow P) \mid \sim \mid (a \rightarrow Q) \quad (\rightarrow - \text{dist})$$

$$\lambda x:A. a \rightarrow (P \mid \sim \mid Q) = (\lambda x:A. a \rightarrow P) \mid \sim \mid (\lambda x:A. a \rightarrow Q) \quad (\text{input-dist})$$

Take care!

- 1 Ensure that you are using the correct laws when distributing
- 2 Consider the process

$$P \mid \sim \mid (Q \mid R)$$

- 3 This process may either behave as P, or as the choice of Q and R

Take care (2)!

- 1 Now consider the process

$$(P \mid \sim \mid Q) \sqcap (P \mid \sim \mid R)$$

- 2 Our distributive law for $\sqcap$ expands this to

$$(P \sqcap P) \mid \sim \mid (P \sqcap R) \mid \sim \mid \\ (Q \sqcap P) \mid \sim \mid (Q \sqcap R)$$

Take care (3)!

$$\begin{array}{c} (P \sqcup P) \mid \sim \mid (P \sqcup R) \mid \sim \mid \\ (Q \sqcup P) \mid \sim \mid (Q \sqcup R) \end{array}$$

- 1 The first bracket reduces to P using $\sqcup$ -idem

$$P \mid \sim \mid (P \sqcup R) \mid \sim \mid (Q \sqcup P) \mid \sim \mid (Q \sqcup R)$$

- 2 Every behaviour of the second and third bracket is possible of the other two: $(P \sqcup R)$ rejects the behaviours of Q , and $(Q \sqcup P)$ rejects the behaviours of R
- 3 Every subsequent behaviour is one of P , Q , or R

Take care (4)!

- ① Therefore we may conclude that $| \sim |$ distributes over $[]$

$$P \mid \sim \mid (Q \mid [] \mid R) = (P \mid \sim \mid Q) \mid [] \mid (P \mid \sim \mid R)$$

($| \sim | - [] - \text{dist}$)

- ② Question: can you think of an analogous law in boolean logic?

Remember STOP?

- 1 Unit laws are where a repeated application of the law makes no difference
- 2 We met the process STOP in lecture 1
 - This process does nothing

$$\text{STOP} \ [] \ P = P \qquad ([] - \text{unit})$$

- 3 Note: there is no need to consider a right unit law ($P \ [] \ \text{STOP} = P$) as we can easily infer this using symmetry

Table of Contents

- 1 Introduction
- 2 Algebra, laws, and semantics
- 3 But what about verification?
- 4 The semantics of CSP
- 5 Algebraic laws
- 6 Summary**

Summary

- 1 In this seminar, we were introduced to some of the algebraic laws of CSP. These laws allow us to manipulate processes into equivalent processes. We noted how many of the laws look similar in nature to laws we have seen in other branches of mathematics.
- 2 In the next lecture, we will look at one of the semantic models of CSP—process traces

Summary

- ➊ Advance reading: Schneider, pages 85–131
- ➋ Advance reading: Roscoe, pages 183–194
- ➌ Advance reading: Woodcock and Davies, chapter 5